Plan & Workload Distribution - PJT1

Plan-

Phase 1: Secure Segmented CAN-FD Architecture and Configurable Key Transport

Timeline: August 1 – August 25

Objective

Implement a segmented CAN-FD in-vehicle network using virtual interfaces and representative vehicle domains connected through a Central Gateway. Establish device trust, remote attestation, secure session establishment, and protected runtime communication using configurable cryptographic and transport components.

Core Architectural Workflow

1. Secure Enrollment and Credential Provisioning

- Establish a trusted enrollment process for each ECU and the Central Gateway.
- Generate or reconstruct an ECU-specific device-root secret using the selected hardware-rooted identity approach.
- Derive separate credentials for transport protection, signing-key protection, and startup-control protection.
- Store only the required ECU-specific transport credential and public verification information at the Gateway.
- Keep sensitive ECU credentials protected and unavailable when the device is powered down.

2. Authenticated Boot and Remote Attestation

- The Gateway initiates each startup session with fresh session context and challenge material.
- The Gateway protects startup-control messages against unauthorized triggering or modification.
- The ECU validates the Gateway challenge and produces a post-quantum-capable attestation response.
- The attestation binds ECU identity, session context, freshness information, protocol version, and challenge content.
- The Gateway verifies the response using the ECU's enrolled verification credential.

3. Indexed Large-Payload Transfer

- Transfer large attestation objects using a configurable indexed block-transfer mechanism.
- Include transfer-session information, fragment ordering, status indicators, and payload data in each network frame.

- Maintain receiver-side reassembly state, duplicate detection, freshness validation, and bounded timeouts.
- Support selective retransmission of missing blocks through authenticated recovery requests.
- Apply retry limits and state validation to prevent recovery abuse and resource exhaustion.

4. Secure Session Establishment

- After successful attestation, both parties derive a temporary transport-protection key bound to the active session and attestation transcript.
- The Gateway generates a fresh operational session secret.
- Deliver the session secret using authenticated protection under the temporary transport key.
- Require a protected ECU confirmation before accepting the session as established.
- Keep the precise key-derivation method, authenticated-encryption mechanism, and frame layout configurable.

5. Protected Runtime Telemetry

- Define runtime frames as protected communication units with configurable telemetry capacity.
- Apply confidentiality, integrity, authenticity, and replay protection to runtime data.
- Use independent traffic protection for each communication direction.
- Construct unique per-message protection inputs from session context, sender identity, direction, and monotonic message state.
- Reject stale, repeated, malformed, or unauthenticated messages.

Phase 1 Milestones and Deliverables

- Functional segmented CAN-FD architecture connected through a Central Gateway.
- Successful ECU device-root reconstruction.
- Authenticated startup and remote-attestation workflow.
- Reliable indexed transfer and selective recovery of large attestation data.
- Secure session establishment and confirmation.
- Protected runtime telemetry exchange with replay resistance.
- Configuration documentation covering the selected security, transport, and frame-allocation choices.

Phase 2: Gateway-Side Federated Learning IDS Integration

Timeline: August 26 – September 12

Objective

Extend the secured CAN-FD framework with a lightweight Federated Learning-based Intrusion Detection System. The system detects abnormal communication behavior and bus attacks while keeping raw vehicle telemetry local to each participating vehicle environment.

Core Architectural Workflow

1. Gateway-Side Inspection Strategy

- Execute intrusion detection inside the trusted Gateway after message verification and decryption.
- Treat invalid protected frames as security events rather than valid application traffic.
- Ensure encrypted payload data is not directly used for content-based anomaly detection.

2. Feature Engineering and Data Preparation

- Prepare automotive intrusion data to match the project's protected telemetry schema.
- Create structured time-series features from decrypted telemetry, message identity, timing behavior, ordering behavior, authentication failures, and recovery events.
- Maintain configurable feature selection so the IDS can adapt to the final telemetry format.

3. Model Development and Edge Inference

- Develop a lightweight anomaly-detection model suitable for Gateway-side execution.
- Train the model using local vehicle data and simulated attack data.
- Export the selected model to a portable edge-inference format.
- Integrate local inference directly into the Gateway runtime.

4. Federated Learning Integration

- Deploy multiple simulated vehicles, each with an independent CAN-FD network and Gateway.
- Train local models using local telemetry and security events.
- Aggregate model updates through a federated-learning workflow without centralizing raw vehicle data.
- Protect model-update communication between vehicle environments and the aggregation service.

5. Threat Emulation

- Simulate replay attempts, message manipulation, authentication failures, flooding, malformed fragmentation traffic, and recovery-message abuse.
- Measure local detection behavior for both valid anomalies and invalid protected traffic.
- Record detection performance and operational overhead.

Phase 2 Milestones and Deliverables

- Gateway-side inspection of verified and decrypted telemetry.
- Local anomaly detection integrated into the Gateway runtime.
- Federated aggregation across simulated vehicle environments.
- Privacy-preserving model-update workflow.
- Detection accuracy, false-positive behavior, inference cost, and resource-use measurements.

Phase 3: Integrated System Evaluation and Comparative Benchmarking

Timeline: September 13 – September 23

Objective

Integrate the complete architecture and evaluate reliability, latency, computational cost, bus impact, security properties, and implementation trade-offs.

Core Architectural Workflow

1. Full Lifecycle Consolidation

Validate the complete sequence:
Device-root reconstruction
→ authenticated startup
→ remote attestation
→ indexed transfer and selective recovery
→ secure session establishment
→ protected runtime telemetry
→ Gateway-side intrusion detection

2. Performance and Resource Profiling

- Measure execution time and resource use for each startup and runtime operation.
- Measure fragmentation, reassembly, recovery, session establishment, protected communication, and local IDS inference.
- Record CPU usage, memory use, throughput, latency, and communication overhead.
- Separate virtual-network software measurements from modeled and physical bus measurements.

3. Reliability and Fault Testing

- Introduce controlled loss, duplication, corruption, reordering, stale-session, timeout, and recovery-abuse conditions.
- Measure recovery success, retry behavior, invalid-message rejection, and resilience under communication faults.
- Validate that attacks do not cause uncontrolled retransmission or resource exhaustion.

4. Comparative Trade-Off Analysis

- Compare indexed transfer with a selected baseline transport approach.
- Compare the implemented session-establishment approach with a deferred alternative key-establishment approach.
- Compare runtime communication overhead with intrusion-detection overhead.
- Document security, latency, reliability, implementation complexity, and deployment trade-offs.

Phase 3 Milestones and Deliverables

- Integrated end-to-end software demonstration.
- Benchmark tables for timing, resource use, reliability, and recovery.
- Network-load and timing visualizations.
- Controlled-fault test results.
- Security-performance trade-off analysis.
- Rationale for the final selected implementation configuration.

Phase 4: Hardware-in-the-Loop Validation

Timeline: September 24 – October 10

Objective

Deploy the consolidated system onto physical embedded hardware and validate communication timing, transport reliability, power use, and end-to-end behavior over a real CAN-FD network.

Core Architectural Workflow

1. Hardware Deployment

- Deploy the Gateway and ECU applications to embedded processing platforms.
- Connect the processing nodes through compatible CAN-FD controllers, transceivers, and physical bus wiring.
- Preserve the software architecture and configurable protocol behavior used during simulation.

2. Physical Network Configuration

- Configure the target bus timing, message priorities, network segmentation, and transport parameters.
- Ensure safety-relevant traffic receives priority over startup, attestation, diagnostic, and experimental traffic.

3. Physical Protocol Validation

- Execute device-root reconstruction, authenticated startup, remote attestation, indexed transfer, selective recovery, session establishment, protected telemetry, and Gateway-side IDS inference.
- Measure real hardware communication behavior and processing overhead.
- Validate reliability under controlled physical-network load and fault conditions.

4. Power and Energy Analysis

- Measure baseline power use and workload-specific energy consumption.
- Compare the energy cost of startup security operations, runtime protection, and IDS inference.
- Document measurement assumptions and sensor limitations.

Phase 4 Milestones and Deliverables

- Operational embedded CAN-FD testbed.
- Successful physical execution of the complete secure communication lifecycle.
- Hardware timing, transport-reliability, and processing-overhead results.
- Power and energy profiles for the main system workloads.
- Final comparison between simulation and hardware-validation results.

Workload Distribution-

Kaustuv (23BDS1060) : Infrastructure, Network & Hardware, Docker/SocketCAN setup, 60/4 fragmentation logic, system profiling, and physical Raspberry Pi integration.
Kunal (23BDS1097) : Cryptography & Security Protocol, PUF emulation, ML-DSA, HKDF logic, ASCON encryption, and threat simulation.
Tapan (23BDS1151) : Machine Learning & Data Architecture, Federated Learning, LSTM/ONNX inference, telemetry feature engineering, and data analysis.

Phase 1: Secure Segmented CAN-FD & Key TransportThe foundational network and crypto layer.

Kaustuv: Sets up the Docker and SocketCAN (vcan) environments. Also responsible for writing the heavy network logic for the 60/4 indexed block-transfer protocol (handling the 41-frame fragmentation, bitmaps, and NACK selective retransmission).
Kunal: Writes the native cryptographic functions. Handles the SRAM-PUF fuzzy extractor emulation, ML-DSA attestation integration, HKDF derivations, and the ASCON-AEAD session wrapping.
Tapan: Defines the exact byte-level schema for the 64-byte protected runtime frames and builds the extraction hooks at the Gateway so decrypted telemetry and protocol metadata can cleanly feed into the IDS feature vector in Phase 2.

Phase 2: Gateway-Side FL-IDS IntegrationThe AI and threat detection layer.

Kaustuv: Scales his Phase 1 Docker setup to simulate the multi-vehicle V2X fleet required for the Federated Learning aggregation.
Kunal: Acts as the Red Team. Leads the threat emulation using can-utils, actively trying to break the network with replay attempts, ASCON-tag corruption, and NACK abuse to test Tapan's IDS model.
Tapan: Leads this phase. Handles the data preparation (ROAD/SynCAN), trains the LSTM Autoencoder, exports it to ONNX, and implements the local edge inference inside the Gateway. Also writes the Flower (flwr) federated averaging logic.

Phase 3: Integrated Evaluation & BenchmarkingThe testing and profiling layer.

Kaustuv: Leads the computational profiling. Runs cProfile, perf, and tcpdump to capture raw CPU, memory, and latency metrics across the consolidated system.
Kunal: Executes the reliability and controlled fault testing. Programmatically drops, duplicates, and reorders packets to prove the 60/4 burst recovery and ASCON counters hold up under stress.
Tapan: Handles the comparative trade-off analysis. Will take Kaustuv and Kunal's raw logs, aggregate the data, and build the security-versus-performance mathematical models (e.g., mapping the IDS latency against the baseline CAN constraints).

Phase 4: Hardware-in-the-Loop ValidationThe physical embedded deployment layer.

Kaustuv: Takes points on the hardware testbed. Wires the Raspberry Pis, interfaces the MCP2518FD SPI-to-CAN controllers, configures the 500kbps/2Mbps physical bus bitrates, and integrates the INA219 current sensors.
Kunal: Migrates the cryptography scripts to the Pi hardware, specifically tuning the ML-DSA attestation execution and HKDF transport to ensure SPI bottlenecks don't break the timing requirements.
Tapan: Manages the live execution of the ONNX model on the physical Gateway, ensuring edge inference stays under the 10ms real-time constraint. Will also log and analyze the physical power and energy metrics generated by the hardware testbed.